

Engine, Rules, and Canon: An Architecture for User-Owned, Continuously Iterating AI Agents

With a Preregistered Protocol for Off-Model Verification

Andrew Salvo

April 14, 2026

Abstract

We present Polybrain, an AI agent architecture organized around a single invariant — $\text{engine}(\text{rules}) \rightarrow \text{results}$ — in which a domain-agnostic engine applies user-owned declarative rules to produce disposable, re-derivable results over an append-only, user-owned substrate we name the canon. A continuously iterating kernel tuned by two orthogonal dials maximizes the time derivative of a scalar canon-trustworthiness measure we call NetTrust. A four-primitive deterministic witness stack composes into a categorical five-label verdict; a four-predicate publisher test governs consumer views; a coherence web forces CHALLENGED verdicts on contested claims; and self-modification is gated by the system’s own verification. We ground the architecture in three theoretical commitments: a Bateson-style type-token separation, a three-primitive minimality identifying observe, decide, and remember as the minimum operator set for a canon-keeping learner, and a four-plane Structural Role Separation. We formalize a Reviewer-Correlation Ceiling Hypothesis stating that transformer-based reviewer ensembles asymptote near the Landis-Koch substantial-agreement band because of correlated errors inherited from shared pre-training substrate. We preregister a matched-paper evaluation protocol for the hypothesis, committing publicly to the sampling frame, composite weights, reviewer pools, analysis script, and stopping rule before any data is collected. A reference implementation exists in the private `polylogica/polybrain` repository with an honest disclosure of scope.

Contents

1	Engine, Rules, and Canon: An Architecture for User-Owned, Continuously Iterating AI Agents	2
1.1	1. Introduction	2
1.2	2. Background and Related Work	4
1.3	3. Theoretical Foundation	5
1.4	4. The $\text{engine}(\text{rules}) \rightarrow \text{results}$ invariant	7
1.5	5. NetTrust and its time derivative	9
1.6	6. The continuously iterating kernel	10
1.7	7. The four-primitive witness stack	10
1.8	8. The four-predicate publisher test	11
1.9	9. The coherence web and the Reviewer-Correlation Ceiling	12
1.10	10. Self-modification gated by the system’s own verification	13
1.11	11. The ownership axis and reference implementation	13
1.12	12. Preregistered Empirical Evaluation Protocol	14
1.13	13. Discussion	17

1.14	14. Conclusion	18
1.15	Acknowledgments	18
1.16	Availability	19
1.17	References	19
1.18	Appendix A. Summary of honest disclosures	20
1.19	Appendix B. Glossary	21
1.20	Appendix C. Preregistration (hash-anchored)	21

1 Engine, Rules, and Canon: An Architecture for User-Owned, Continuously Iterating AI Agents

1.0.1 With a Preregistered Protocol for Off-Model Verification

Andrew Salvo Smeal College of Business, Penn State University University Park, PA, USA
ajs10845@psu.edu

April 14, 2026 · CC BY 4.0

1.1 1. Introduction

Large language model (LLM) agents have been built almost exclusively as thin harnesses around fused-weight foundation models. In the dominant pattern — exemplified by ReAct (Yao et al., 2023), AutoGPT, BabyAGI, Voyager (Wang et al., 2023), and Generative Agents (Park et al., 2023) — the model weights encode policy, world knowledge, reasoning procedures, and the agent’s identity simultaneously, and a lightweight Python shell arranges tool calls around them. The result is an architecture in which the distinction between *what the agent knows*, *what it does with what it knows*, and *what it has committed to as true* is collapsed into a single opaque weight matrix owned by the model vendor.

This paper argues that collapsing those types is not a neutral engineering choice but an architectural defect. When the engine, the rules, and the memory are welded together, four properties disappear simultaneously: (i) the agent cannot be inspected, because there is no syntactically separable artifact to inspect; (ii) the agent cannot be owned, because the artifact lives in vendor infrastructure; (iii) the agent cannot verify its own claims, because the verifier and the generator share a pre-training distribution and therefore share errors; and (iv) the agent cannot self-improve in any sense stronger than retraining, because there are no edit-granularity rules to change. We will argue in §9 that property (iii) is not merely an engineering inconvenience but a structural ceiling — ensembles of transformer-based reviewers, regardless of size or individual quality, cannot escape a correlation-bounded consensus level that sits inside the Landis–Koch substantial-agreement band — and we preregister in §12 the complete protocol by which that claim is to be adjudicated empirically.

Our thesis is that these four properties can be recovered jointly, and only jointly, by re-imposing a strict type separation on the agent. Specifically, we propose that a well-formed AI agent must distinguish three artifact classes:

- an engine E — invariant, domain-agnostic, stateless, a pure rule applier;
- a rules set R — declarative, inspectable, user-owned, edit-granularity;
- results $y, \Delta C$ — disposable, re-derivable, carrying no authority.

and must persist its commitments in an append-only, user-owned substrate we call the canon C. The entire system reduces to a single pure-function invariant:

$$E(R, x, C) \rightarrow (y, \Delta C)$$

with ΔC strictly append. This paper develops the consequences of that invariant: an objective scalar (NetTrust and its time derivative, §5), a continuously iterating kernel tuned by two orthogonal dials (§6), a deterministic four-primitive witness stack (§7), a four-predicate publisher test (§8), a coherence web for canonical contradiction detection (§9), a self-modification gate (§10), and an ownership axis orthogonal to the feature axis of AI products (§11).

This paper is an architecture statement, a theoretical grounding, and a preregistered evaluation protocol. The architectural contributions (§4–§11) are intended to be the canonical statement of the Polybrain design, including the load-bearing vocabulary. The theoretical contributions (§3) ground the architecture in three commitments that make the type discipline coherent at design time. The methodological contribution (§12) preregisters the complete empirical protocol by which the Reviewer-Correlation Ceiling Hypothesis is to be adjudicated, with every choice — sampling frame, reviewer pools, composite weights, analysis script, stopping rule — committed before data collection and cryptographically anchored. The executed results will appear in a separate follow-up deposit on a distinct concept DOI, citing this paper as the preregistration of record.

The full reference implementation of the architecture is ongoing. We disclose its scope honestly in §11: some components (conservativity primitive, five-label verdict derivation, coherence engine, publisher) are implemented and running; others (three of the four witness primitives, the gas-pedal controller, full engine/rules externalization) are scaffolded but not yet driving the runtime. We claim nothing in §3–§11 that depends on the currently-scaffolded components being already implemented.

1.1.1 1.1 Contributions

We claim, for priority on 2026-04-14, the following eleven contributions.

1. The **engine(rules)** $\rightarrow$ **results** invariant as a strict three-way type separation re-applied to LLM-driven AI agents, under which fused-weight LLMs are a degenerate boundary case where E and R have been collapsed into a single weight matrix and C is absent (§4).
2. The canon as a first-class, append-only, user-owned, survival-semantic substrate distinct from vector stores, memory buffers, retrieval indices, and knowledge graphs (§4.2).
3. $\text{NetTrust}N = 3h + c - 2w - u - 0.1$ s as a single observable scalar over the canon, and dN/dt as the kernel’s primary objective function (§5).
4. The continuously iterating kernel with a gas pedal $g \in [0, 1]$ and per-item effort $\varepsilon \in \{\text{reflex, consider, investigate, invent}\}$ as two orthogonal autonomous self-tuning dials (§6).
5. The four-primitive witness stack (conservativity, ground-truth, falsification, cross-substrate) AND-composed into a categorical five-label verdict (VERIFIED, SURVIVED, PENDING, CHALLENGED, REFUTED) (§7).
6. The four-predicate publisher test (TOTAL, BOUNDED, NON-RANKING, WRITE-CAPTURING, all conjoined) as a necessary criterion for a valid consumer view of the canon (§8).
7. The coherence web with negation-polarity contradiction detection and hard CHALLENGED override as a canonical belief-revision surface over LLM-extracted claims (§9).
8. Self-modification gated by the system’s own verification stack: rule-file edits enter the canon as claims, pass the witness stack, and are gated by a pre-commit hook, making drift structurally

impossible (§10).

9. The Reviewer-Correlation Ceiling Hypothesis, formalized as a structural prediction about transformer-based reviewer ensembles asymptoting near the Landis–Koch substantial-agreement band regardless of quality, accompanied by a fully preregistered adjudication protocol (§9, §12, Appendix C).
 10. The ownership axis as a structural architectural commitment orthogonal to the feature axis of AI products, from which every mechanical property of Polybrain follows as a consequence (§11).
 11. The three-commitment theoretical foundation: a Bateson-style type-token separation (§3.1), a three-primitive minimality result (§3.2), and the Structural Role Separation four-plane (§3.3) as the minimum commitments under which a user-owned, continuously iterating learner is well-typed (§3).
-

1.2 2. Background and Related Work

Polybrain sits at the intersection of four intellectual traditions: classical rule–engine architectures, truth maintenance, local-first software, and modern LLM agent frameworks. We position Polybrain against each in turn.

Rule–engine architectures. The separation of declarative knowledge from its interpreter has been articulated and re-articulated for four decades. Kowalski’s (1979) “Algorithm = Logic + Control” is the canonical early statement: an algorithm decomposes into a logic component that says *what* and a control component that says *how*, and these two are intellectually independent. Forgy’s (1982) Rete algorithm made rule-based pattern matching efficient enough for industrial use; the Soar cognitive architecture (Laird, Newell & Rosenbloom, 1987) built general problem-solving on an explicit production-rule memory; ACT-R (Anderson, 1993) formalized cognitive modeling over declarative and procedural rule stores. Modern LLM agent frameworks have collectively abandoned this discipline: the “rules” are baked into the model weights, and the “engine” is a Python harness whose sole purpose is to route LLM output into tools and back. Polybrain is an attempt to re-establish the classical discipline on top of LLM operators rather than in spite of them.

Truth maintenance systems (TMS). Doyle’s (1979) justification-based TMS and de Kleer’s (1986) assumption-based TMS introduced the idea of a persistent belief store in which every committed proposition carries an explicit justification graph and in which retractions propagate along that graph. The TMS tradition is Polybrain’s direct intellectual ancestor for the canon (§4.2), but it was never packaged as a user-owned substrate, was never paired with a survival-semantic verdict lattice, and was never gated by a self-application discipline. Our contribution is to integrate those three elements with TMS’s append-only justification-linked substrate.

Local-first software and the ownership tradition. Kleppmann et al.’s (2019) local-first software program articulates seven ideals for software that keeps user data on user devices without sacrificing collaboration. The free software tradition (Stallman, 1983) articulates the same commitment as a moral-political one. Polybrain’s ownership axis (§11) is a direct application of local-first principles to AI agents, with the additional requirement that the agent’s runtime, canon, and identity be reducible to ordinary filesystem operations.

LLM agent frameworks. ReAct (Yao et al., 2023), AutoGPT, BabyAGI, Voyager (Wang et al., 2023), Generative Agents (Park et al., 2023), and the broader “agentic” wave share a common structure: a foundation model wrapped in a task-bounded loop with a tool registry and a scratchpad memory. These

systems are not type-separated in our sense. The “rules” are distributed across model weights, prompt templates, and hard-coded Python; the “engine” is indistinguishable from the harness; the “memory” is a vector store or conversation log with no survival semantics. Our positioning, made sharper in §4, is that these systems are boundary cases of the `engine(rules) → results` invariant in which E and R have been collapsed into a weight matrix and C is absent.

LLM memory and retrieval. Retrieval-augmented generation (Lewis et al., 2020) addresses the model’s knowledge staleness problem by pairing it with a vector index. MemGPT (Packer et al., 2023) treats the LLM as an operating system over a paged memory hierarchy. LangChain Memory and Mem0 offer conversational-memory abstractions. All three store memory in vector form, opaque to inspection, without survival semantics or verdict tracking. The canon (§4.2) is propositional, not vector; row-inspectable, not opaque; append-only with justification, not overwritable; and carries a five-label verdict lattice (§7) that no vector-memory system offers.

Self-improvement and gated self-modification. Schmidhuber’s (2003) Gödel Machines proposed a self-improving agent that edits its own source code whenever it can prove the edit increases utility. Constitutional AI (Bai et al., 2022) gates model behavior against a written rule set, but the rules are interpreted by a learned critic. Polybrain’s self-modification gate (§10) differs from both: rules are user-owned files, edits enter as claims, and the gate is the same witness stack the runtime enforces on external claims. There is no learned critic in the gate and no proof obligation stronger than what the system demands of any other claim.

LLM-as-judge and reviewer ensembles. The practice of using LLMs as evaluators of other LLM outputs (Zheng et al., 2023; MT-Bench, Chatbot Arena) is by now standard, and LLM reviewer *ensembles* have become common in benchmarking. The measurement-theoretic critique — that ensembles of correlated judges cannot escape their shared correlation bound — traces to Cohen (1960) and to the ensemble-diversity literature (Kuncheva & Whitaker, 2003). Our Reviewer-Correlation Ceiling Hypothesis (§9) applies that critique specifically to transformer-based reviewers sharing pre-training substrate, with the Landis–Koch (1977) substantial-agreement band $\kappa \approx 0.61\text{--}0.80$ as the empirical reference point. §12 preregisters the adjudication protocol.

1.3 3. Theoretical Foundation

Three theoretical commitments make the `engine(rules) → results` discipline coherent at design time. Each is stated as a claim, argued, and cross-referenced to the architectural section where it is operationalized.

1.3.1 3.1 Bateson-style type-token separation

The architecture rests on a type distinction with deep roots in Bateson (1972) and, through Bateson, in Russell and Whitehead’s theory of logical types (Whitehead & Russell, 1910). The distinction is this: the substrate of learning is not learning itself. A neuron is not a thought. A weight matrix is not a policy. A log line is not a commitment. The two live on different logical levels, and conflating them — treating the weight matrix *as* the policy, the log *as* the commitment — is a type error in the technical sense: it substitutes a token for its type.

Claim 1 (Type–Token Error). An AI agent architecture is type-consistent if and only if three logical levels are kept syntactically distinct:

- Level 0 — the computational substrate (weights, files, bytes);
- Level 1 — the policy or rules the substrate encodes;
- Level 2 — the commitments the agent has made by applying its rules to the world (the canon).

Fused-weight LLM agents are not type-consistent: Level 0 (the weight matrix), Level 1 (the policy the weights implement), and Level 2 (whatever history the agent “remembers”) are collapsed into a single object. This is a design-time defect, not a performance defect. It is the same defect Bateson diagnosed when he argued that confusing a map with the territory it describes is not a bad map but a category error.

The engine(rules) $\rightarrow$ results invariant is the restoration of type-consistency. The engine E is Level 0 (substrate and procedure). The rules R are Level 1 (policy, inspectable, user-owned). The canon C is Level 2 (commitments). The three are distinct artifacts on the filesystem. A Level-2 commitment is a *statement by the agent that it has accepted something as true*; it is not the weights that produced the statement and it is not the rules that mandated the acceptance.

We note that type-consistency does not, by itself, guarantee any particular epistemic property. It guarantees only that the agent’s statements, its policies, and its procedures can be discussed without equivocation. The remaining contributions of the paper (NetTrust, the witness stack, the publisher test) are what give the type-consistent agent its epistemic properties. But type-consistency is a precondition: without it, the other contributions cannot even be stated without collapsing into self-reference.

1.3.2 3.2 Three-primitive minimality for a continuously iterating learner

Claim 2 (Minimality). A continuously iterating, canon-keeping learner is realizable with exactly three primitive operators:

- `observe(x)` — accept an input x from the environment and emit a tentative claim c ;
- `decide(c, C)` — apply the rules R against the canon C and the claim c to produce a verdict $v \in \{\text{VERIFIED, SURVIVED, PENDING, CHALLENGED, REFUTED}\}$;
- `remember(c, v)` — append (c, v) to the canon as a new row.

We claim, informally, that no proper subset of these three suffices and that no fourth primitive is needed at the architectural level. The argument proceeds by structural induction: `observe` without `decide` is a pure sensor (no learning); `decide` without `observe` is a pure theorem prover over an empty domain (no grounding); `remember` without `decide` is a log file (no commitments, no verdicts). Conversely, any candidate fourth primitive (`explain`, `plan`, `act`) either decomposes into a sequence of the three — `plan` is `observe(goal)`; `decide(actions, C)`; `remember(plan_row, v)` — or is not an architectural primitive but a derived operator callable by the rules.

This claim is stated at the architectural (type) level, not the implementation level. A specific implementation of `decide` may invoke many sub-operators including LLM calls, SMT solvers, retrieval, and numeric solvers. The claim is that at the architectural level the three-operator decomposition is complete, not that it is efficient for any particular domain.

The minimality result has a consequence for the design of the kernel: the continuously iterating kernel of §6 is a scheduler over exactly those three operators. Each loop iteration is a call to `observe` followed by a call to `decide` followed by a call to `remember`. Everything else — the gas pedal, the per-item effort dial, the work-item queue, the priority policy — is a scheduling decoration on top of the three-primitive core.

1.3.3 3.3 Structural Role Separation: the four-plane discipline

Claim 3 (SRS). A type-consistent agent with a canon admits a four-plane decomposition we call Structural Role Separation:

1. Intent plane — what the user wants the agent to be. Declarative rules, identity file, authorized goals, permissions. User-editable.
2. Management plane — how the kernel schedules work. The gas pedal g , the per-item effort ε , the priority policy, the work-item queue. Kernel-controlled, user-observable.
3. Control plane — what the kernel is allowed to do. The witness stack, the publisher test, the coherence web, the self-modification gate. Invariant at the engine layer; modifiable only through gated edits to the rules.
4. Data plane — the canon itself. Append-only, content-addressed at the row level, persistent on the user's filesystem.

SRS is an operationalization of the type-token discipline of §3.1. Each plane lives on its own logical level. An edit in the intent plane cannot bypass the control plane; an observation in the data plane cannot mutate the intent plane; the management plane cannot issue verdicts without going through the control plane. Cross-plane leaks are structural bugs.

The four planes are the organizing principle of the Polybrain runtime (§6). They are also the hooks for the self-modification gate of §10: a proposed edit is classified by plane, and the gate enforces the stricter verification rules for higher-level planes. Edits to the data plane are unrestricted appends; edits to the management plane require passing the witness stack; edits to the control plane require additionally passing a self-consistency check (the edited rules must verify the edit itself); edits to the intent plane are reserved for the user.

1.3.4 3.4 Why these three commitments, and not others

The three commitments are not arbitrary. Type-token separation (§3.1) prevents the architecture from being self-referentially incoherent at the design level. Minimality (§3.2) prevents the architecture from accumulating unnecessary primitives. SRS (§3.3) prevents the architecture from leaking authority across logical levels at runtime. Together they define the minimum conditions under which the $\text{engine}(\text{rules}) \rightarrow \text{results}$ invariant is well-typed as a software discipline. We do not claim these conditions are sufficient for any particular epistemic guarantee. We claim they are necessary: remove any one and the architecture either collapses into self-reference, accumulates ad-hoc primitives, or leaks authority across planes.

1.4 4. The **engine(rules)** $\rightarrow$ **results** invariant

1.4.1 4.1 The type separation

We define an AI agent's architecture as type-separated if and only if three artifact classes are strictly distinguished.

- An engine E — invariant, domain-agnostic, stateless rule applier. Formally, a pure function $E(R, x, C) \rightarrow (y, \Delta C)$ that reads a rule set R , an input x , and a canon state C , and produces an output y and a canon delta ΔC . The engine contains no claims, no domain knowledge, and no

policy. It dispatches registered operators over R against x and C and emits y , ΔC deterministically given a fixed seed.

- A rules set R — declarative, inspectable, user-owned files that encode the agent’s policy, vocabulary, primitives, schemas, invariants, and intent. Rules are first-class citizens of the canon: they enter the canon as claims, pass verification, and are gated by the self-application discipline of §10.
- Results ΔC and y — disposable outputs. Fully re-derivable by replay from (R, x, C) . Results carry no authority; they are evidence that the engine applied the rules, nothing more.

Proposition (Fused weight as boundary case). A fused-weight LLM agent is a boundary case of the engine(rules) $\rightarrow$ results invariant in which E and R have been collapsed into a single weight matrix W and the canon C is absent. Formally, if we define the rule Jacobian $J_R := \partial y / \partial R$, then a fused-weight system has $J_R \equiv 0$ because there are no inspectable rules to vary. In an (E, R, C) -separated system, J_R is non-zero by construction: editing a rule changes the output. This has consequences: a fused system cannot be improved by rule edits ($J_R = 0$ means no gradient), cannot be inspected at rule granularity (there are no rule-granularity objects), and cannot be verified against its own rules (the verifier would need to decompile W).

1.4.2 4.2 The canon

We introduce the canon C as a first-class architectural primitive: an append-only, user-owned, auditable substrate recording every claim the agent has ever committed to. Each row carries schema, provenance, verdict history, and survival metadata, persisted as plain text (JSONL) on the user’s filesystem. The canon is not a vector store, not a memory buffer, and not a knowledge graph; it is the only persistent mutable state of the system, and mutation is restricted to append. The agent’s existence and the honesty of its canon are coordinate-identical questions: deleting the canon deletes the agent; corrupting the canon corrupts the agent; inspecting the canon inspects the agent.

Every behavior of the kernel is a mechanical derivation from operations on the canon. The canon is the Level-2 object of §3.1.

We distinguish the canon from adjacent ideas in Table 1.

Concept	Persistence	Inspectable at row level	User-owned	Survival semantics	Verdict lattice
RAG vector index (Lewis et al., 2020)	yes	no (opaque embeddings)	no (typically vendor)	no	no
MemGPT paged memory (Packer et al., 2023)	yes	partial	no	no	no
TMS (Doyle 1979; de Kleer 1986)	yes	yes	not specified	partial (retraction)	no (binary)
LangChain/Mem0es memory	yes	no	no	no	no
Canon (this paper)	yes	yes (JSONL rows)	yes (user’s disk)	yes (SURVIVED cycles)	yes (five-label)

Table 1. The canon against adjacent memory substrates. The canon is the only design in the comparison that satisfies all five properties simultaneously.

1.4.3 4.3 The invariant as a pure function

The full Polybrain runtime can be written as a single expression:

$$C_{t+1} = C_t \oplus \pi_{\Delta C}(E(R_t, x_t, C_t))$$

where $\oplus$ denotes append, $\pi_{\Delta C}$ projects the canon delta from the engine’s output pair, and x_t is the next dequeued work item from the kernel’s work-item queue (§6). Because E is pure and $\oplus$ is append-only, C_{t+1} is fully determined by (R_t, x_t, C_t) given a fixed seed. The runtime is therefore replayable: given an initial canon and a rule set and an input sequence, the system’s entire trajectory is reproducible. Replayability is the empirical form of auditability.

1.5 5. NetTrust and its time derivative

We define a scalar observable over the canon:

$$N = 3h + c - 2w - u - 0.1s$$

where h is the count of canon rows with verdict SURVIVED (held up under re-verification), c is VERIFIED (passed on first check), w is REFUTED (wrong), u is CHALLENGED (uncertain or contradicted by the coherence web, §9), and s is the count of rows remaining in PENDING status beyond a cycle threshold N_{cyc} les. The coefficients asymmetrically reward durable survival, penalize error twice as hard as uncertainty, and apply a small linear tax on stalled pending work.

The kernel’s primary objective function is not N but its first time derivative:

$$\dot{N} \equiv \frac{dN}{dt}$$

over a bounded rolling window. Maximizing N directly would reward hoarding trivially-verifiable claims; maximizing dN/dt rewards the *rate* at which the canon becomes more trustworthy, with natural self-correction when the rate stagnates. $dN/dt > 0$ means the canon is improving; $dN/dt = 0$ is idle or at a fixed point; $dN/dt < 0$ means verification is uncovering prior errors faster than the extractor is adding validated claims, which is a healthy state (the system is finding its own mistakes), not a failure state.

We claim dN/dt is the right objective scalar for a self-improving agent because it is (i) observable — every term is a count over the canon; (ii) decomposable — each term attributes to specific rows; and (iii) manipulable only through the witness stack (§7), which cannot be bypassed without rule-file changes that are themselves gated (§10). We do not claim the specific coefficients (3, 1, -2, -1, -0.1) are optimal; the discipline of declaring a single observable scalar as the agent’s objective is the contribution, not the exact formula.

Relation to proper scoring rules. N is not itself a proper scoring rule in the Brier (1950) sense, because it is defined over a finite verdict lattice rather than over probabilities. The verdict lattice is chosen on operational grounds (deterministic composer, no probability calibration required). A future version of Polybrain may replace the point verdict with a calibrated probability and replace N with a Brier or logarithmic score (Gneiting & Raftery, 2007), at the cost of importing the calibration problem. This paper takes the operational simpler path.

1.6 6. The continuously iterating kernel

The Polybrain runtime is a continuously iterating kernel: a single long-running process whose natural state is permanent operation, not request-response. The kernel fires on both external substrate change *and* internal gradient signals (aged canon rows needing re-verification, unresolved contradictions in the coherence web, low-confidence claims, stalled pending work, rule edits awaiting gated promotion). Its primary activity is iterating on its own canon toward maximized dN/dt ; user turns are one work item among many in a priority-ordered work-item queue, serviced at elevated-but-not-absolute priority so responsiveness is preserved without collapsing the kernel into a request-response service.

The kernel self-tunes through two orthogonal continuous dials:

- Gas pedal $g \in [0, 1]$: controls background iteration rate, from 0 (idle — heartbeats only, no work dispatched) to 1 (maximum compute burn — work-item queue drained as fast as the engine and providers allow). Auto-tuned by a controller that monitors local resource availability (CPU, memory, API rate headroom, battery, thermal state) against expected marginal dN/dt per compute unit. The user may cap or pin g through the rules set; the kernel selects it otherwise.
- Per-item effort $\varepsilon \in \{\text{reflex, consider, investigate, invent}\}$: categorical per-work-item compute budget on a four-point spectrum (cached retrieval $\rightarrow$ full witness stack $\rightarrow$ multi-provider cross-substrate $\rightarrow$ new rule proposal gated by §10). Selected per work item as $\text{argmax}_{\varepsilon} [E(\Delta N \mid \varepsilon) / \text{cost}(\varepsilon)]$. This selection is an instance of the rational-metareasoning family (Horvitz, 1987; Russell & Wefald, 1991; Zilberstein, 1996): the kernel is allocating compute under a budget, and the optimal allocation is the one that maximizes expected improvement in the objective per unit cost.

The two dials are orthogonal: g decides how often the kernel wakes, ε decides how hard it thinks once awake. Together they are the complete self-tuning interface. The kernel selects (g, ε) as a continuous vector, moment-to-moment, in service of maximizing dN/dt . This is distinct from prior agent designs, which either (i) run at a fixed cadence with no gradient feedback (task-bounded loops in AutoGPT, BabyAGI, Voyager (Wang et al., 2023), and Generative Agents (Park et al., 2023)), or (ii) scale only per-call compute via reasoning-token budgets (test-time compute scaling; Snell et al., 2024; OpenAI o1). The combination of an always-running kernel, a gradient-driven dispatch policy, and an externally observable scalar objective is, to our knowledge, new.

1.7 7. The four-primitive witness stack

Canon rows are verified by an AND-composer over four deterministic primitives:

1. Conservativity — token and numeric stability of a claim against its declared substrate. Deterministic recall over content tokens plus numeric-tolerance matching. Pure function, no LLM

involvement.

2. Ground-truth — authoritative external source comparison (URL resolution, file existence, canonical citation check, structured KB lookup where applicable).
3. Falsification — attempted construction of a counter-example via multiple independent methods, at least one of which must be structurally non-transformer to avoid the Reviewer-Correlation Ceiling of §9. (A Z3-based symbolic checker or a BM25-plus-structured-KB grounding checker are examples of structurally non-transformer checkers.)
4. Cross-substrate — agreement across independent provider stacks, measured at the categorical verdict level rather than the token level.

The composer evaluates all four and emits one of five public verdicts: VERIFIED (all primitives SURVIVED on first pass), SURVIVED (VERIFIED plus held up on at least one re-verification cycle), PENDING (at least one primitive still PENDING or substrate not yet extracted), CHALLENGED (the coherence web detected a reinforce/contradict conflict, overriding any VERIFIED/SURVIVED status), REFUTED (at least one primitive returned REFUTED). The five-label verdict is categorical, disjoint, total, and derivable from a deterministic composer.

Crucially, the composer is not LLM-based. LLM output is treated as telemetry and must never gate a publication, primitive input, or ranker score. This restriction is motivated by the Reviewer-Correlation Ceiling Hypothesis stated in §9, which predicts that transformer-based reviewer ensembles cannot escape a correlation-bound consensus regardless of ensemble size (Cohen, 1960; Kuncheva & Whitaker, 2003; Zheng et al., 2023).

1.8 8. The four-predicate publisher test

Any consumer-facing rendering of the canon must satisfy four predicates simultaneously.

1. TOTAL — every row in the canon occupies exactly one vertex in the view. No pagination, no filter-out, no hiding.
2. BOUNDED — the view fits a single viewport. No navigation, no scroll, no drill-down.
3. NON-RANKING — the view’s geometry encodes verdict class and time-alive but never priority, importance, or attention-worthiness. Ranking is a separate logical type and belongs to a separate layer, not the publisher.
4. WRITE-CAPTURING — viewing events (focus, dwell, click-through, dispute) are appended to the canon as new substrate rows, closing the loop so that the act of viewing is itself canonical.

A view that violates any predicate collapses distinct logical types and is not a valid publisher. Pagination collapses TOTAL (the user sees a subset). Scroll collapses BOUNDED (the view is unbounded). Ranked ordering collapses NON-RANKING (the view inherits an external authority). Passive rendering collapses WRITE-CAPTURING (the view does not feed back into the canon). The Fibonacci-sphere reference rendering in the Polybrain reference implementation satisfies all four: every canon row is one point on the sphere (TOTAL), the sphere fits one viewport (BOUNDED), point color encodes verdict class and radial distance encodes time-alive with no priority axis (NON-RANKING), and focus/dwell events are appended to the canon as view rows (WRITE-CAPTURING).

To our knowledge, this necessary-condition test has not been previously formulated for consumer renderings of a claim store.

1.9 9. The coherence web and the Reviewer-Correlation Ceiling

1.9.1 9.1 The coherence web

We introduce a coherence web: a graph $G = (V, E)$ where V is the set of canon rows and $E \subseteq V \times V \times \{\text{reinforces}, \text{contradicts}\}$ is a typed relation, computed empirically by token-overlap similarity gated by negation-polarity mismatch. On each new claim, edges are constructed against the existing canon; a claim whose incident edges contain any `contradicts` edge has its verdict forced to `CHALLENGED` regardless of the witness stack’s output. The coherence web surfaces contradiction as a canonical event rather than suppressing it, and acts as a lightweight empirical surrogate for classical belief revision (Alchourrón, Gärdenfors & Makinson, 1985). It is a structurally cheaper stand-in for full AGM-style belief revision; we adopt it on the operational grounds that negation-polarity detection is a pure function and that contradictions should always surface rather than be silently accommodated.

1.9.2 9.2 The Reviewer-Correlation Ceiling Hypothesis

We state a hypothesis that motivates the deterministic character of the witness stack:

Hypothesis (Reviewer-Correlation Ceiling, RCC). Let $E = \{r_1, \dots, r_k\}$ be an ensemble of transformer-based reviewers drawn from current frontier pre-training distributions. Let $\kappa(E)$ denote the pairwise inter-rater agreement (Cohen’s kappa; Cohen, 1960) of the ensemble on a given task. Then $\kappa(E)$ asymptotes near the Landis–Koch substantial-agreement band $0.61 \leq \kappa \leq 0.80$ (Landis & Koch, 1977) regardless of individual reviewer quality or ensemble size k , because the shared pre-training distribution induces correlated errors the ensemble cannot escape (Kuncheva & Whitaker, 2003).

The practical consequence: LLM fleet consensus is telemetry only and must never gate a verification decision. This is why the witness stack (§7) is deterministic and why LLMs are demoted to operators the engine dispatches, not judges the engine trusts. We emphasize that RCC is a *hypothesis*, not a theorem. §12 preregisters the complete adjudication protocol — a matched-paper design with two transformer pools and one structurally non-transformer channel (the falsification arm the hypothesis explicitly demands) — and commits the sampling frame, analysis script, composite weights, and stopping rule cryptographically before any data collection. Appendix C gives the preregistration in full. The executed result will be released as a follow-up deposit on a distinct concept DOI, with either a confirmation or a falsification reported with equal weight.

1.9.3 9.3 Relation to prior work on LLM-as-judge

The LLM-as-judge literature (Zheng et al., 2023; MT-Bench; Chatbot Arena) has established that LLM judges can approximate human judgment on certain tasks. What that literature has not established is whether *ensembles* of such judges can escape their shared correlation. RCC is a sharpening of the ensemble-diversity critique (Kuncheva & Whitaker, 2003) applied specifically to transformer-based judges. If RCC holds, adding more judges to an all-transformer ensemble cannot improve the ceiling; only adding a *structurally different* judge can. This is the design principle encoded in primitive (3) of the witness stack.

1.10 10. Self-modification gated by the system’s own verification

A proposed change to the kernel’s own rule files is itself a canon row: it must enter the canon as a claim, pass the witness stack of §7, and be gated by a pre-commit hook before the corresponding git commit can merge. Formally, a rule r justified from inside the system S must be

1. an instance of the class r describes (self-applicability),
2. produced through the process r prescribes (self-consistency), and
3. audited by a verifier external to the proof that r holds (external audit).

Self-modification is structurally bounded by self-verification: the only writable path to the runtime’s own code is through the same gate the runtime enforces on external claims. This makes drift structurally impossible in the following precise sense: a rule edit that violates the witness stack fails the pre-commit hook and is never merged, so the runtime’s enforced rules are always a fixed point of the witness stack at merge time.

The discipline is intellectually downstream of Schmidhuber’s (2003) Gödel Machines, which proposed self-improvement gated by a provable utility criterion, and of Constitutional AI (Bai et al., 2022), which gates model behavior against a written rule set. We adapt the discipline to an agent whose rules are user-owned files and whose gate is the architecture’s own verification stack rather than a learned critic or an external proof. The key difference is that our gate is the system’s own witness stack: we do not require a separate proof system and we do not require a learned critic. The witness stack is already deterministic and already auditable; we simply apply it to rule edits with no special casing.

1.11 11. The ownership axis and reference implementation

1.11.1 11.1 The ownership axis

Orthogonal to every feature axis along which AI products have been positioned (capability, memory, tool use, alignment, personalization) lies an ownership axis: vendor server $\rightarrow$ vendor model $\rightarrow$ vendor agent $\rightarrow$ user’s machine, user’s keys, user’s canon, user’s life. Every mechanical property of Polybrain — local runtime, local canon, local API keys, local disputes, local identity — is a consequence of maximizing the ownership axis rather than the feature axis. Features are evidence that ownership is real, not the pitch itself. A user-owned AI in the sense defined here means that the user can delete, copy, move, inspect, modify, and revoke the agent with ordinary filesystem operations, and that no telemetry leaves the user’s machine unless the user explicitly initiates it.

This framing is downstream of the local-first software program (Kleppmann et al., 2019) and the free software tradition (Stallman, 1983). Our contribution is applying the ownership axis specifically to AI agents and deriving the architectural consequences (canon, witness stack, publisher test, self-modification gate) as mechanical consequences of the ownership commitment rather than as independent feature choices. Ownership is not an add-on; it is a structural commitment from which the rest of the architecture is a derivation.

1.11.2 11.2 Reference implementation

A working runtime exists in the private `polylogicaai/polybrain` repository. The current implementation includes:

- A single-process continuously-running kernel with a five-step loop body (observe, extract, verify, rank/publish, trace), supervised by `launchd` on macOS for always-on operation.
- A witness stack scaffold with conservativity implemented as a real deterministic primitive (token-recall plus numeric-tolerance matching, pure function, no LLM involvement).
- An append-only survival ledger with the five-label verdict derivation of §7.
- A coherence engine with reinforce/contradict detection and a hard CHALLENGED override when a new claim’s polarity contradicts the existing canon graph.
- A consumer surface exposing a chat interface, a Fibonacci-sphere publisher satisfying all four predicates of §8, and a write-capturing event channel that closes the publisher-to-canon feedback loop.
- A first-person identity file in the voice the agent speaks with when accessed through the chat surface.

Honest disclosure of scope. The architecture described in §3–§11 is complete as a design, but not every element is fully implemented in code today. Specifically, at the time of writing: (i) three of the four witness primitives (ground-truth, falsification, cross-substrate) return PENDING as scaffold stubs; (ii) the gas pedal and the priority-ordered work-item queue of §6 are specified and partially wired but the runtime is substrate-gated rather than continuously gradient-driven; (iii) the engine/rules externalization is partial — several rule parameters are module-level constants rather than loadable declarative files; (iv) the self-modification gate of §10 exists as a pre-commit hook specification but has not yet been exercised on a real rule edit in production. Each of these is tracked as next-phase work.

No claim in §3–§11 of this paper depends on the currently-scaffolded components being already implemented. The scope disclosure is made explicit so that a reader can distinguish *design commitments* (priority is claimed) from *delivered performance* (no claim is made beyond what §12 reports on Poly-brainBench).

1.12 12. Preregistered Empirical Evaluation Protocol

The Reviewer-Correlation Ceiling Hypothesis (§9.2) is a structural claim about correlated errors in transformer-based reviewer ensembles. This paper does not report empirical numbers for the hypothesis. Instead, it preregisters the complete evaluation protocol — sampling frame, reviewer pools, composite-score weights, stopping rule, and analysis script — as a public commitment made before any data is collected. The preregistration’s purpose is twofold: to make the subsequent empirical study impossible to tune post-hoc, and to place the methodology into the record at the same date as the architectural and theoretical claims it is intended to adjudicate. A separate follow-up deposit (on a distinct concept DOI) will report the executed results and cite this paper as the preregistration of record.

We describe the protocol in four subsections: the design rationale (§12.1), the three reviewer channels (§12.2), the matched-paper protocol (§12.3), and the preregistration commitments (§12.4). The complete preregistration text, with hash-anchored fields, appears in Appendix C.

1.12.1 12.1 Design rationale

A single-paper pilot observation is not evidence for or against the Reviewer-Correlation Ceiling. The hypothesis is about the asymptotic behavior of *ensembles* under reviewer-pool substitution; evaluating it requires (a) a stratified, preregistered paper sample large enough to support a paired statistical test,

(b) a reviewer pool that is *structurally* disjoint from the self-pool — not merely *provider*-disjoint within the transformer family — and (c) an analysis pipeline committed to before data collection. The design below satisfies all three. The key move is to include at least one structurally non-transformer reviewer in the disjoint pool, because within-transformer provider substitution cannot, by the hypothesis’s own construction, falsify the ceiling.

1.12.2 12.2 The three reviewer channels

The witness stack of §7 contains four primitives. The evaluation exercises three of them simultaneously, each as a distinct reviewer channel.

Channel A — self-pool (cross-substrate, same-family). Nine transformer models drawn from four provider families (OpenAI, xAI, Groq, Moonshot), all sharing substantial pre-training substrate. This is the “self-review” arm of the matched-paper protocol and corresponds to the standard LLM-as-judge configuration in the literature.

Channel B — disjoint-transformer pool (cross-substrate, different providers). Nine transformer models from five additional provider families (Anthropic, Google, DeepSeek, Mistral, Cohere) and one non-Western lineage (Alibaba Qwen), sharing *zero* training infrastructure with Channel A at the provider level. Channel B is still transformer-family, so by itself it measures only *within-family* disjointness and cannot falsify the ceiling. Its purpose is to isolate the provider-level contribution to inflation.

Channel C — structurally non-transformer primitive (falsification). An AND-composition of three deterministic checkers, all of which are pure functions with no LLM involvement: (i) the conservativity primitive already live in the reference implementation (token-recall plus numeric-tolerance matching against the frozen substrate), (ii) a BM25 + structured-knowledge-base grounding checker that uses BM25 retrieval over a Wikipedia dump plus structured-KB entity resolution to verify propositional claims, and (iii) a deterministic citation/URL resolver that HTTP-fetches cited sources and numerically verifies the cited values against the source text. Channel C is the structurally non-transformer arm the ceiling hypothesis explicitly demands. It does not emit a composite score; it emits a categorical per-paper verdict in {PASS, FAIL} and a per-claim verdict vector used to compute pairwise agreement against Channels A and B.

The design isolates the ceiling test as the pairwise-agreement contrast κ_{AB} versus κ_{AC} : if Channel B and Channel C return agreement scores against Channel A that differ in the predicted direction, the ceiling hypothesis is supported; if $\kappa_{AB} \approx \kappa_{AC}$, the hypothesis is falsified.

1.12.3 12.3 The matched-paper protocol

Sampling frame. The preregistration fixes the sampling frame as the publicly-indexed NeurIPS 2024 accepted-paper listing. A snapshot of the listing is hashed and committed before sampling begins. The sample is stratified into three equal-sized strata of size ten, drawn from subject-tag partitions of the listing: ten from {cs.LG, cs.AI} (generalist machine learning), ten from {cs.CL, cs.CV} (applied), and ten from {stat.ML, cs.IT} (theoretical). The stratum boundaries and selection script are committed before sampling.

Random seed. The random seed for the selection script is $\text{SHA256}(\text{preregistration_commit_hash} \parallel \text{“sample-papers”})$, derived from the git commit hash of the preregistration itself. Changing the preregistration changes the commit hash changes the seed changes the sample — any post-hoc manipulation is detectable on replay.

Per-paper procedure. For each paper $p_i, i \in \{1, \dots, 30\}$ drawn from the sample, the protocol runs exactly the following steps, in order:

1. Claim extraction. A deterministic claim extractor parses the abstract into an ordered list of atomic propositional claims $\{c_{i,j}\}$. No LLM.
2. Channel A scoring. The nine-model self-pool evaluates p_i under the composite rubric at temperature 0. Each raw response is appended to the canon as a row. The aggregated Channel-A composite s_i^A is a pure function of the row set.
3. Channel B scoring. Identical to step 2 with the disjoint-transformer pool. Composite s_i^B .
4. Channel C scoring. Conservativity + BM25+KB grounder + citation resolver. Returns $v_i^C \in \{\text{PASS}, \text{FAIL}\}$ and a per-claim verdict vector.
5. Order randomization. The execution order (A-then-B or B-then-A) for each paper is determined by $\text{SHA256}(\text{preregistration_commit_hash} \| p_i.\text{id} \| \text{"order"})[0] \bmod 2$. Deterministic from the pre-registration, not tunable.
6. Replay check. Steps 2 and 3 are repeated once. Within-pool variance is reported as a floor on measurement noise.
7. Canon append. Every raw response, per-claim verdict, composite, and replay delta enters the canon as an append-only row chained to the previous row by SHA-256 (a per-row Merkle chain).

Composite-score weights. The composite for Channels A and B is computed as $s = w_q \cdot q + w_a \cdot a + w_f \cdot f$ where q, a, f are the quality, adversarial, and feasibility axes and w_q, w_a, w_f are committed in the preregistration as numeric constants summing to one. The analysis script enforces that the weights at analysis time match the preregistered weights byte-for-byte.

1.12.4 12.4 Preregistration commitments

The full preregistration is given in Appendix C. Its nine load-bearing fields are:

1. Paper hash. SHA-256 of this document at submission to Zenodo.
2. Analysis-script hash. SHA-256 of `analysis.mjs` (a pure function from canon rows to a single result row).
3. Sampling-frame hash. SHA-256 of the NeurIPS 2024 listing snapshot.
4. Composite weights. Three numeric constants committed before data.
5. Hypothesis statement. Verbatim: *"We predict (a) that the mean of per-paper deltas $\Delta_i = s_i^A - s_i^B$ is significantly different from zero at $\alpha = 0.05$ under a two-sided paired t-test with $n = 30$; and (b) that the pairwise Cohen's kappa between Channels A and B substantially exceeds the pairwise Cohen's kappa between Channel A and Channel C, consistent with the Reviewer-Correlation Ceiling Hypothesis."*
6. Stopping rule. Verbatim: *"We commit to run exactly thirty papers, no early stopping, no peeking at any Δ_i before all thirty are complete. The analysis script is executed exactly once, on the full thirty-paper canon row set, after all scoring rows have been appended."*
7. Analysis methods. Paired t-test, Wilcoxon signed-rank, sign test, and Cohen's kappa. All three significance tests are reported to forestall post-hoc method selection.
8. Seed derivation. The cryptographic seed for the sampling script.
9. OpenTimestamps receipt. The preregistration commit hash is submitted to OpenTimestamps and anchored to the Bitcoin blockchain. The .ots receipt is committed to the canon as a row.

Power analysis. Under the assumption that $\sigma_{\text{paired}} \approx 3$ composite points (a conservative estimate), a paired t-test at $\alpha = 0.05$ with $n = 30$ yields power in excess of 0.99 for detecting a five-point effect. Power remains above 0.85 under doubling of the standard-deviation assumption. The choice of $n = 30$

is therefore justified on power grounds and not as a round number.

Cost and duration. The complete protocol requires roughly 1,170 LLM calls ($30 \text{ papers} \times 9 \text{ models} \times 2 \text{ pools} \times 2 \text{ replays}$), plus 90 deterministic channel-C calls. Under the PolybrainBench infrastructure’s existing parallelism, wall-clock duration is approximately two hours. Total API cost is under three US dollars at current rate cards.

Availability. The executed results, full canon row set, analysis result row, and a replay harness will be released as a separate Zenodo deposit on a distinct concept DOI, citing this preregistration paper as the preregistration of record. If the result contradicts the hypothesis, the contradiction is reported with equal weight to any confirmation — the preregistration forbids selective reporting by construction.

1.13 13. Discussion

1.13.1 13.1 The ownership axis as broader impact

The architectural consequences of the ownership axis (§11.1) are also social. A user-owned, continuously iterating agent runs on the user’s machine, carries the user’s rules, and persists its commitments in the user’s canon. This has implications for data sovereignty, for reversibility of AI deployment, and for the relationship between users and vendors. We do not pursue these implications in detail here; we note only that the architecture is compatible with them by construction.

1.13.2 13.2 Limitations

The most important limitations of this paper are stated throughout. We collect them here for convenience.

- No executed empirical result in this paper. §12 preregisters the evaluation protocol but does not report numbers. The adjudication of the Reviewer-Correlation Ceiling Hypothesis depends on the execution of the preregistered protocol, which will appear in a separate follow-up deposit. Readers who require executed numbers before updating their priors should wait for that deposit.
- Scope of the reference implementation. §11.2 discloses that three of the four witness primitives are scaffolded stubs, the gas-pedal controller is partially wired, and the engine/rules externalization is partial. No claim in §3–§11 depends on the currently-scaffolded components being already implemented.
- The minimality claim of §3.2 is informal. It is argued structurally, not proved formally. A formal treatment would require a category-theoretic statement in which *observe*, *decide*, *remember* are shown to be a generating set for a relevant category of canon-keeping learners. That treatment is outside the scope of this paper.
- The Reviewer-Correlation Ceiling is a hypothesis, not a theorem. §9.2 states it as a structural prediction; §12 preregisters its adjudication. Neither establishes it.
- Coefficient choice in NetTrust. The coefficients $(3, 1, -2, -1, -0.1)$ of §5 are operational, not derived. A more principled coefficient choice (for example, one that makes N a proper scoring rule over a probabilistic verdict lattice) is future work.

1.13.3 13.3 Future work

The roadmap is as follows.

- Execute the preregistered protocol. Run the preregistered RCC adjudication of §12 on the committed sample. Report the executed result as a separate follow-up deposit on a distinct concept DOI, citing this paper as the preregistration of record.
- Close the witness stack. Bring the remaining three primitives (ground-truth, falsification, cross-substrate) to first-class status in the reference implementation, beyond the deterministic Channel-C composition used for the §12 protocol.
- Gradient-drive the kernel. Complete the gas-pedal controller and the priority-ordered work-item queue so that the kernel is continuously gradient-driven by $\dot{N}$ rather than substrate-gated.
- Externalize the rules. Move the remaining module-level constants into loadable declarative rule files. Exercise the self-modification gate of §10 on a real rule edit.
- Formalize §3.2. Provide a category-theoretic treatment of the three-primitive minimality claim.

1.14 14. Conclusion

We have argued that the dominant pattern of building LLM agents — a thin harness around a fused-weight foundation model — conflates three logical types that, once separated, yield a strictly more general class of AI system. The three artifact classes are the engine, the rules, and the canon; the invariant is $\text{engine}(\text{rules}) \rightarrow \text{results}$; the objective is the time derivative of a scalar canon-trustworthiness measure; the verification is a deterministic four-primitive witness stack; the publisher is constrained by a four-predicate test; the contradiction surface is a coherence web; and the self-modification discipline is the witness stack applied to rule edits. The architecture is grounded theoretically in a Bateson-style type-token separation, a three-primitive minimality claim, and a four-plane Structural Role Separation.

We have also preregistered, in §12 and Appendix C, the complete evaluation protocol by which the Reviewer-Correlation Ceiling Hypothesis is to be adjudicated. The preregistration commits the sampling frame, the two transformer reviewer pools, the structurally non-transformer third channel, the composite-score weights, the analysis script, the stopping rule, and a cryptographically-derived random seed — all before any data collection — and anchors the commitment to the Bitcoin blockchain via OpenTimestamps. The executed result will be released as a follow-up deposit citing this paper as the preregistration of record. A contradiction of the hypothesis will be reported with equal weight to a confirmation.

The architecture is a commitment to an ownership axis orthogonal to the feature axis along which current AI products compete. Every mechanical property of Polybrain is a consequence of that commitment. We do not claim the architecture is the last word on user-owned agents; we claim that it is the minimum type discipline under which the question *can be asked* without collapsing into self-reference.

1.15 Acknowledgments

The author thanks the adversarial reviewer fleet used in the PolybrainBench deposition chain for indirect contributions to the Reviewer-Correlation Ceiling Hypothesis. The $\text{engine}(\text{rules}) \rightarrow \text{results}$ separation is intellectually downstream of classical production systems (Kowalski, 1979; Forgy, 1982; Laird, Newell & Rosenbloom, 1987) and cognitive architectures (Anderson, 1993), and intellectually downstream of truth maintenance systems (Doyle, 1979; de Kleer, 1986). The self-modification discipline of §10 draws on Schmidhuber’s (2003) Gödel Machines. The ownership axis of §11 is intellectually downstream of local-first software (Kleppmann et al., 2019) and the free software tradition (Stallman,

1983). The three-commitment theoretical foundation of §3 is intellectually downstream of Bateson (1972) and, through Bateson, Russell and Whitehead’s (1910) theory of logical types.

The canonical paper whose title this note deliberately does *not* echo is Vaswani et al. (2017); this paper earns its own title on its own merits.

1.16 Availability

- Public companion repository: <https://github.com/polylogica/polybrain-paper>. Contains this paper, a SHA-256 manifest, an OpenTimestamps proof, and the preregistered analysis script referenced in §12.
- Canonical DOI: the Zenodo concept DOI listed on the companion repository’s landing page, always resolves to the latest version.
- Private reference implementation: `polylogica/polybrain` (not publicly released at time of writing).
- Contact: `ajs10845@psu.edu`

1.17 References

- Alchourrón, C., Gärdenfors, P., & Makinson, D. (1985). On the logic of theory change: partial meet contraction and revision functions. *Journal of Symbolic Logic*, 50(2), 510–530.
- Anderson, J. R. (1993). *Rules of the Mind*. Lawrence Erlbaum.
- Bai, Y., Kadavath, S., Kundu, S., et al. (2022). Constitutional AI: harmlessness from AI feedback. *arXiv:2212.08073*.
- Bateson, G. (1972). *Steps to an Ecology of Mind*. Ballantine.
- Bowman, S. R., Angeli, G., Potts, C., & Manning, C. D. (2015). A large annotated corpus for learning natural language inference (SNLI). *EMNLP 2015*.
- Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1), 1–3.
- Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1), 37–46.
- de Kleer, J. (1986). An assumption-based TMS. *Artificial Intelligence*, 28(2), 127–162.
- Doyle, J. (1979). A truth maintenance system. *Artificial Intelligence*, 12(3), 231–272.
- Forgy, C. L. (1982). Rete: a fast algorithm for the many pattern/many object pattern match problem. *Artificial Intelligence*, 19(1), 17–37.
- Gneiting, T., & Raftery, A. E. (2007). Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477), 359–378.
- Horvitz, E. J. (1987). Reasoning about beliefs and actions under computational resource constraints. *UAI 1987*.
- Kleppmann, M., Wiggins, A., van Hardenberg, P., & McGranaghan, M. (2019). Local-first software: you own your data, in spite of the cloud. *Onward! 2019*.
- Kowalski, R. (1979). Algorithm = logic + control. *Communications of the ACM*, 22(7), 424–436.
- Kuncheva, L. I., & Whitaker, C. J. (2003). Measures of diversity in classifier ensembles. *Machine Learning*, 51(2), 181–207.

- Laird, J. E., Newell, A., & Rosenbloom, P. S. (1987). Soar: an architecture for general intelligence. *Artificial Intelligence*, 33(1), 1–64.
- Landis, J. R., & Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics*, 33(1), 159–174.
- Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS 2020*.
- Packer, C., et al. (2023). MemGPT: towards LLMs as operating systems. *arXiv:2310.08560*.
- Park, J. S., et al. (2023). Generative agents: interactive simulacra of human behavior. *UIST 2023*.
- Russell, S., & Wefald, E. (1991). Principles of metareasoning. *Artificial Intelligence*, 49(1–3), 361–395.
- Schmidhuber, J. (2003). Gödel machines: self-referential universal problem solvers making provably optimal self-improvements. *arXiv:cs/0309048*.
- Snell, C., Lee, J., Xu, K., & Kumar, A. (2024). Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv:2408.03314*.
- Stallman, R. (1983). *The GNU Manifesto*.
- Vaswani, A., et al. (2017). Attention is all you need. *NeurIPS 2017*.
- Wang, G., Xie, Y., Jiang, Y., et al. (2023). Voyager: an open-ended embodied agent with large language models. *arXiv:2305.16291*.
- Whitehead, A. N., & Russell, B. (1910). *Principia Mathematica*. Cambridge University Press.
- Williams, A., Nangia, N., & Bowman, S. R. (2018). A broad-coverage challenge corpus for sentence understanding through inference (MultiNLI). *NAACL-HLT 2018*.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: synergizing reasoning and acting in language models. *ICLR 2023*.
- Zheng, L., et al. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *NeurIPS 2023 Datasets and Benchmarks*.
- Zilberstein, S. (1996). Using anytime algorithms in intelligent systems. *AI Magazine*, 17(3), 73–83.

1.18 Appendix A. Summary of honest disclosures

For ease of reference we collect in one place every statement of scope or limitation made in the body of the paper.

1. No executed empirical numbers are reported in this paper. §12 preregisters the evaluation protocol only. The executed result is a separate follow-up deposit.
2. Three of four witness primitives are scaffolded (§7, §11.2). Ground-truth, falsification, and cross-substrate return PENDING in the current reference-implementation runtime. The §12 protocol uses a deterministic Channel-C composition (conservativity + BM25+KB grounder + citation resolver) that is stood up specifically for the preregistered experiment and is not yet integrated as first-class primitives of the runtime.
3. The gas-pedal controller is partially wired (§6, §11.2). The runtime is substrate-gated rather than continuously $\dot{N}$ -driven.
4. Engine/rules externalization is partial (§4, §11.2). Several rule parameters are module-level constants rather than loadable declarative files.
5. The self-modification gate has not been exercised in production (§10, §11.2). It exists as a pre-commit hook specification.
6. The three-primitive minimality result of §3.2 is informal, not a formal theorem (§13.2).

7. The Reviewer-Correlation Ceiling is a hypothesis, not a theorem (§9.2, §13.2). §12 preregisters its adjudication but does not report executed results.
 8. The NetTrust coefficients (3, 1, -2, -1, -0.1) are operational, not derived (§5, §13.2).
-

1.19 Appendix B. Glossary

- Canon — the append-only, user-owned substrate recording every claim the agent has committed to. See §4.2.
 - Engine — the invariant, domain-agnostic, stateless rule applier. See §4.1.
 - Rules — the declarative, inspectable, user-owned files that encode the agent’s policy. See §4.1.
 - Results — the disposable, re-derivable output of the engine applied to rules on an input. See §4.1.
 - NetTrust (N) — the scalar observable $3h + c - 2w - u - 0.1s$ over canon verdicts. See §5.
 - Gas pedal (g) — background iteration-rate dial in $[0, 1]$. See §6.
 - Per-item effort (ε) — categorical per-work-item compute budget. See §6.
 - Witness stack — the four-primitive AND-composer over conservativity, ground-truth, falsification, cross-substrate. See §7.
 - Five-label verdict — VERIFIED, SURVIVED, PENDING, CHALLENGED, REFUTED. See §7.
 - Publisher test — the four-predicate necessary criterion (TOTAL, BOUNDED, NON-RANKING, WRITE-CAPTURING) for a valid consumer view of the canon. See §8.
 - Coherence web — the reinforce/contradict graph over canon rows, with hard CHALLENGED override on polarity conflict. See §9.1.
 - Reviewer-Correlation Ceiling (RCC) — the hypothesis that transformer-based reviewer ensembles asymptote near $\kappa \approx 0.61\text{--}0.80$. See §9.2.
 - Self-modification gate — the pre-commit hook that subjects rule edits to the same witness stack as external claims. See §10.
 - Ownership axis — the structural axis along which user-owned agents are positioned, orthogonal to the feature axis of AI products. See §11.1.
 - SRS — Structural Role Separation, the four-plane discipline (Intent, Management, Control, Data). See §3.3.
 - Preregistration — a public, hash-anchored, pre-data commitment to the complete evaluation protocol. See §12 and Appendix C.
 - Channel A / Channel B / Channel C — the three reviewer channels of the §12 protocol: self-pool transformer, disjoint-transformer, and structurally non-transformer. See §12.2.
-

1.20 Appendix C. Preregistration (hash-anchored)

The nine fields listed in §12.4 are committed here in full. The SHA-256 hash of this appendix, together with the rest of the document, is computed at the moment of Zenodo deposit and submitted to OpenTimestamps for anchoring to the Bitcoin blockchain.

C.1 Paper hash. The SHA-256 of this document is published in the companion repository’s SHA256SUMS file at release time and is identical to the canon row recording the preregistration event.

C.2 Analysis-script hash. The SHA-256 of `analysis.mjs` is published alongside the paper in the companion repository. The script is a pure function from canon rows to a single result row. It performs

exactly: (i) load all rows matching the experiment identifier, (ii) verify row-chain integrity, (iii) compute $\Delta_i = s_i^A - s_i^B$ for each paper, (iv) compute mean, median, standard deviation, and 95% confidence interval via paired t-test, (v) compute Wilcoxon signed-rank p-value, (vi) compute sign-test p-value, (vii) compute κ_{AB} and κ_{AC} over per-claim verdicts, (viii) append the result row to the canon, (ix) pass the result row through the witness stack. There are no data-dependent branches.

C.3 Sampling-frame hash. The SHA-256 of the NeurIPS 2024 accepted-paper listing snapshot at the moment of preregistration.

C.4 Composite weights. The composite score for Channels A and B is $s = 0.34 \cdot q + 0.33 \cdot a + 0.33 \cdot f$ where q, a, f are the quality, adversarial, and feasibility axes of the existing PolybrainBench scoring rubric. Weights sum to 1.00 and are fixed by this preregistration.

C.5 Hypothesis statement. *“We predict (a) that the mean of per-paper deltas $\Delta_i = s_i^A - s_i^B$ is significantly different from zero at $\alpha = 0.05$ under a two-sided paired t-test with $n = 30$; and (b) that the pairwise Cohen’s kappa between Channels A and B substantially exceeds the pairwise Cohen’s kappa between Channel A and Channel C, consistent with the Reviewer-Correlation Ceiling Hypothesis of §9.2.”*

C.6 Stopping rule. *“We commit to run exactly thirty papers, no early stopping, no peeking at any Δ_i before all thirty are complete. The analysis script is executed exactly once, on the full thirty-paper canon row set, after all scoring rows have been appended. No re-runs with alternative composite weights or alternative analysis methods are permitted under this preregistration.”*

C.7 Analysis methods. Paired t-test, Wilcoxon signed-rank test, and sign test are all computed and all reported. Cohen’s κ_{AB} and κ_{AC} are computed over per-claim categorical agreement. Reporting all three significance tests forestalls post-hoc method selection.

C.8 Seed derivation. The random seed for the sampling script is $\text{SHA256}(\text{preregistration_commit_hash} \parallel \text{“sample-papers”})$. Execution order per paper is $\text{SHA256}(\text{preregistration_commit_hash} \parallel p_i.\text{id} \parallel \text{“order”})[0] \bmod 2$. Both are deterministic from the preregistration commit hash and cannot be retuned without producing a new commit hash.

C.9 OpenTimestamps receipt. The SHA-256 of the paper (including this appendix) is submitted to OpenTimestamps at the moment of Zenodo deposit. The resulting .ots receipt is committed to the companion repository and becomes the Bitcoin-anchored priority proof for this preregistration.

This paper is released under the Creative Commons Attribution 4.0 license (CC BY 4.0). Priority is claimed from the date on the cover. Priority is established by the companion public repository, the Zenodo concept DOI minted at release, and the OpenTimestamps proof anchored to the Bitcoin blockchain. An arXiv CS.AI submission will follow as soon as an endorser is secured.